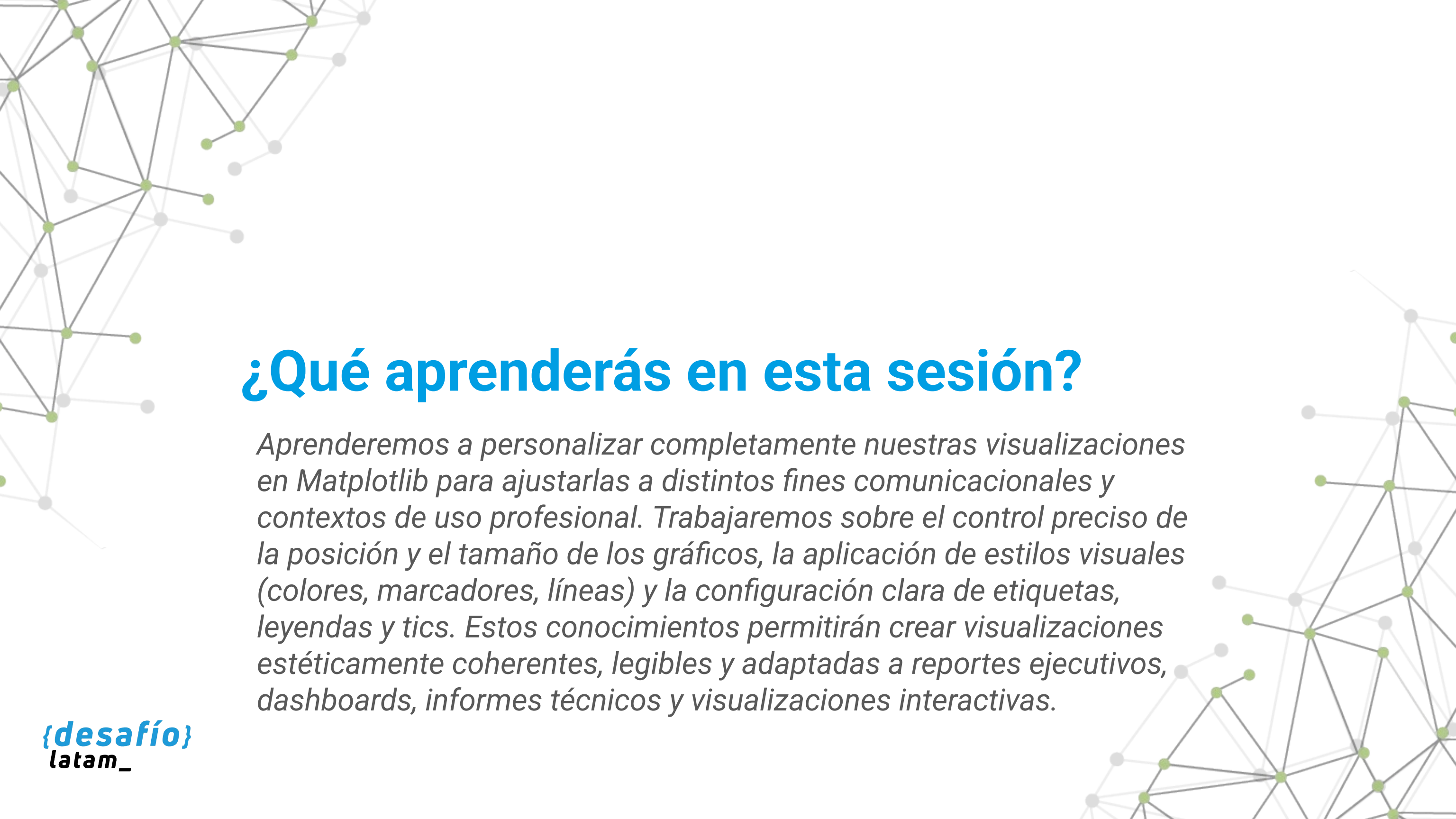




# Visualización de datos

Clase 7 - Personalización avanzada de gráficos en Matplotlib



## ¿Qué aprenderás en esta sesión?

*Aprenderemos a personalizar completamente nuestras visualizaciones en Matplotlib para ajustarlas a distintos fines comunicacionales y contextos de uso profesional. Trabajaremos sobre el control preciso de la posición y el tamaño de los gráficos, la aplicación de estilos visuales (colores, marcadores, líneas) y la configuración clara de etiquetas, leyendas y tics. Estos conocimientos permitirán crear visualizaciones estéticamente coherentes, legibles y adaptadas a reportes ejecutivos, dashboards, informes técnicos y visualizaciones interactivas.*

# Desafío inicial

Trabajas como parte del equipo de análisis de datos de una empresa de logística que debe presentar reportes mensuales a gerencias regionales. El equipo ha recibido observaciones sobre lo poco claros que resultan algunos gráficos enviados en informes anteriores: los tamaños no son proporcionales al espacio del informe, los colores son confusos, los ejes no están bien rotulados y muchas veces no se sabe qué línea corresponde a qué serie de datos.

¿Cómo podemos ajustar con precisión el tamaño, los estilos visuales y los elementos descriptivos de un gráfico para que sea claro, legible y útil en distintos formatos de presentación?

A lo largo de esta sesión exploraremos cómo controlar la posición y el tamaño de nuestras figuras, cómo aplicar estilos coherentes de colores, marcadores y líneas, y cómo configurar adecuadamente los ejes, etiquetas, ticks y leyendas para lograr gráficos informativos y profesionales, adecuados para entornos productivos reales.

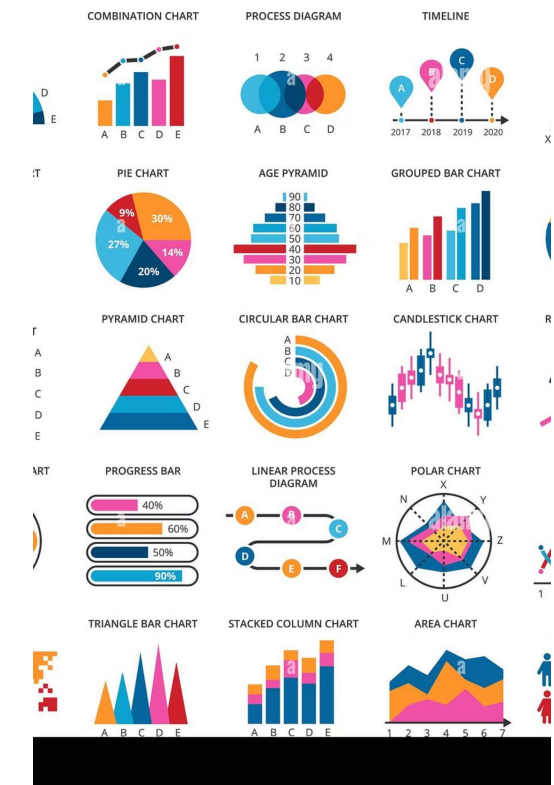
**/\* Posición y tamaño  
de los gráficos \*/**

# Posición y tamaño de los gráficos

**¿Qué tan importante es que un gráfico se ajuste correctamente al espacio disponible en un informe o presentación?**

En entornos productivos, los gráficos rara vez se visualizan aislados.

Normalmente se insertan en documentos PDF, presentaciones, dashboards web o incluso reportes automáticos. Si su tamaño no es proporcional, el texto puede cortarse, los elementos visuales pueden superponerse o la visualización puede volverse ilegible. Por ello, es fundamental controlar la posición y el tamaño de las figuras en Matplotlib desde su creación.



# Controlar el tamaño de la figura

Para establecer el tamaño del gráfico se utiliza el parámetro figsize al crear la figura, el cual define las dimensiones en pulgadas (ancho, alto):

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10, 5)) # 10
pulgadas de ancho, 5 de alto
ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title('Ejemplo de gráfico redimensionado')
plt.show()
```

Nota: Un buen tamaño base para gráficos horizontales simples es entre 8x4 o 10x5, dependiendo del entorno donde se visualizará (pantalla, impresión, presentación).



# Ajuste de márgenes y espacios

En figuras complejas o cuando se agregan muchos elementos (títulos, etiquetas largas, leyendas), los márgenes por defecto pueden resultar insuficientes. Para evitar solapamientos o cortes, se recomienda ajustar los espacios manualmente:

```
plt.tight_layout() # Ajuste automático sin solapamiento  
plt.subplots_adjust(top=0.9) # Ajuste fino: margen superior (de 0 a 1)
```

Estas funciones son especialmente útiles cuando trabajamos con subgráficos o al exportar a formatos fijos (como PNG o PDF).

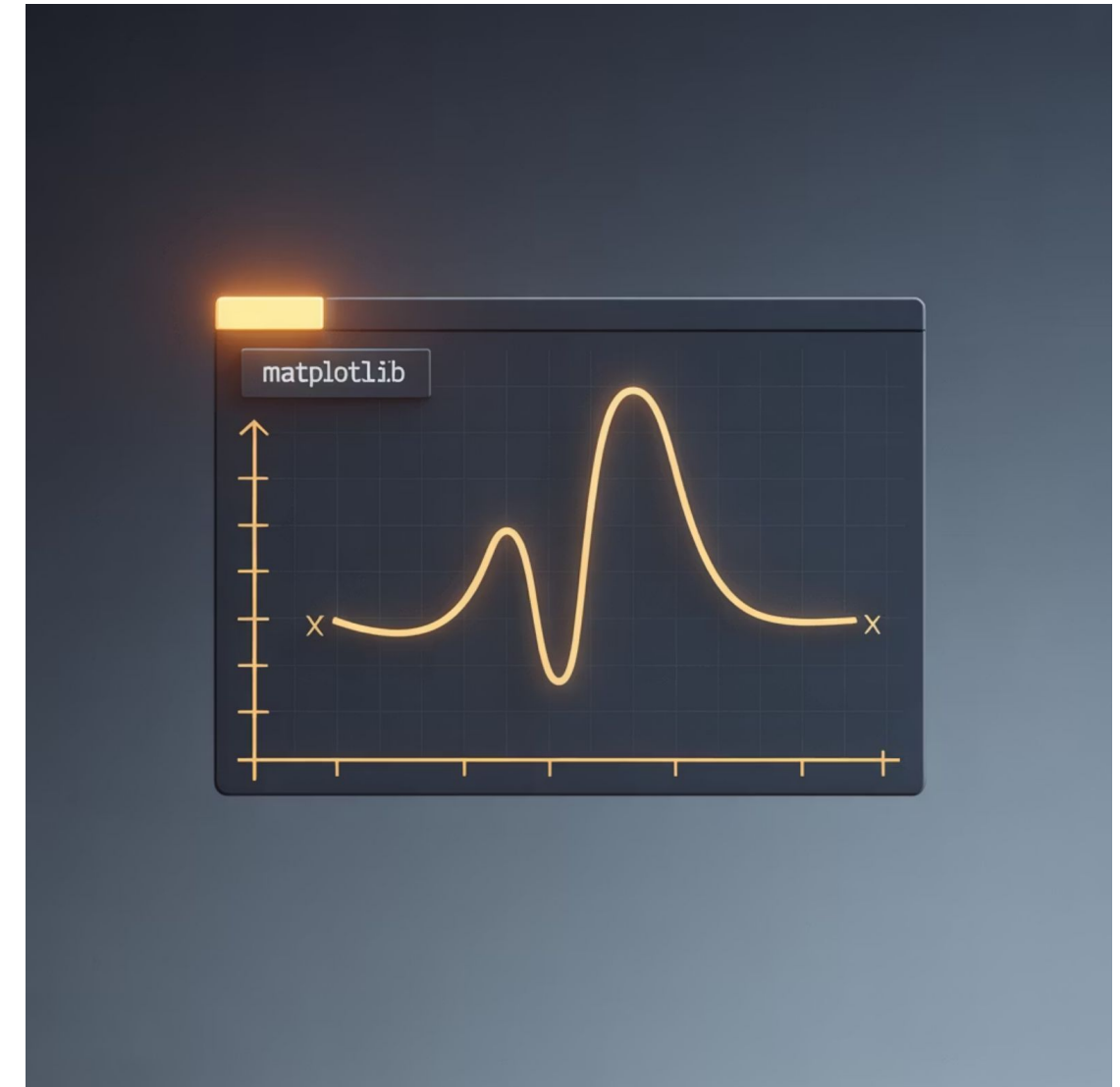


# Posición de los ejes dentro de la figura

Aunque Matplotlib posiciona los ejes de forma automática, podemos definirlos manualmente para un control total usando `fig.add_axes([x0, y0, ancho, alto])`, donde los valores son proporciones relativas dentro de la figura:

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
# Posición y tamaño del eje como % del lienzo
ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title('Posicionamiento manual del gráfico')
plt.show()
```

Este enfoque es útil cuando se necesita superponer visualizaciones, insertar elementos como logos o diseñar layouts personalizados.





# Buenas prácticas

El control de tamaño y posición de gráficos no solo implica saber dibujar una figura; también implica tomar decisiones estratégicas que afectan directamente la claridad y legibilidad del mensaje visual. Aplicar buenas prácticas en este ámbito permite evitar ajustes improvisados, errores de presentación y retrabajo al insertar visualizaciones en contextos como informes técnicos, presentaciones ejecutivas o plataformas de visualización web.



## Definir figsize desde el inicio del gráfico

Evita redimensionamientos posteriores.



## Usar `tight_layout()` siempre que se agreguen textos o subgráficos

Previene errores visuales.



## Controlar márgenes cuando se exportan gráficos

Algunos formatos (como PNG) cortan bordes si no se ajusta `bbox_inches='tight'` al guardar.

# Errores comunes y cómo evitarlos

En la práctica profesional, los errores de presentación en gráficos suelen deberse a descuidos en el dimensionamiento o posicionamiento de los elementos visuales. Estos errores afectan la comprensión del mensaje, especialmente en contextos donde el tiempo o la atención del público son limitados. Identificar y corregir estos problemas es esencial para garantizar que nuestras visualizaciones sean útiles y efectivas.

| Error                        | Causa probable                   | Solución recomendada                    |
|------------------------------|----------------------------------|-----------------------------------------|
| Texto o leyenda cortada      | Tamaño de figura insuficiente    | Aumentar figsize y usar tight_layout()  |
| Solapamiento entre elementos | No ajuste de márgenes            | Aplicar plt.subplots_adjust()           |
| Gráfico se ve deformado      | Relación ancho/alto mal definida | Probar proporciones estándar (ej. 10x5) |

# Dominio del tamaño y posición

Dominar el control del tamaño y la posición de los gráficos permite asegurar que nuestras visualizaciones se integren correctamente en cualquier entorno de presentación. Este dominio es clave para lograr gráficos que no solo sean informativos, sino también profesionalmente presentables.

A continuación, exploraremos cómo aplicar estilos visuales personalizados para potenciar el impacto comunicacional de nuestros gráficos.

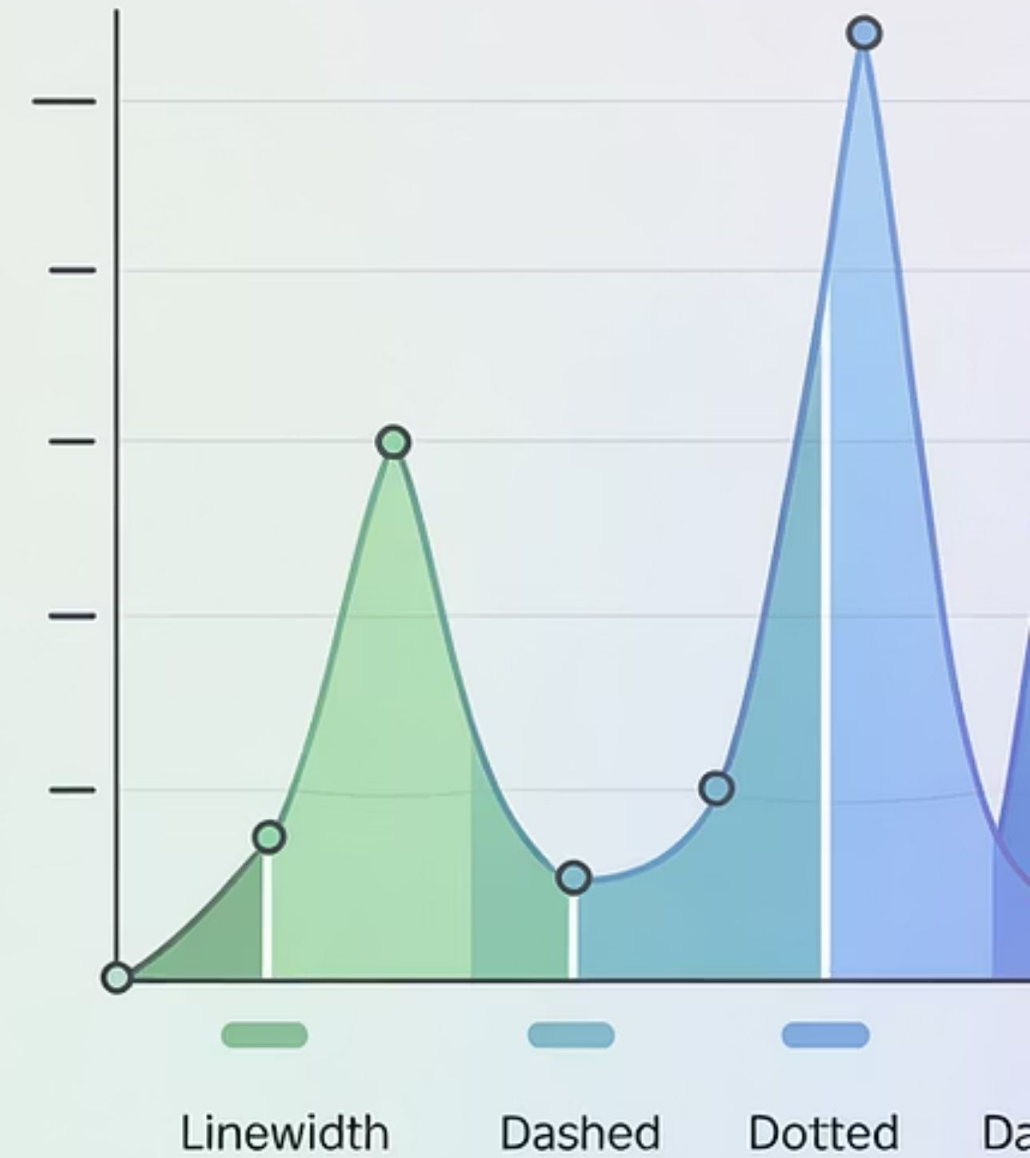


# Colores, marcadores y estilos

## ¿Cómo impacta la elección de colores en la comprensión de un gráfico?

En la visualización de datos, los elementos estéticos no son un mero accesorio: tienen un rol comunicativo fundamental. Colores, estilos de línea y marcadores permiten diferenciar series, destacar patrones y guiar la atención del lector hacia los aspectos más relevantes del gráfico. En contextos profesionales, un gráfico claro y visualmente coherente puede ser la clave para una decisión acertada o para comunicar resultados con impacto.

Matplotlib ofrece múltiples formas de controlar estos estilos, tanto de manera manual como mediante configuraciones globales, lo que permite adaptar la visualización al público, al medio (pantalla, impresión) y al objetivo del análisis.

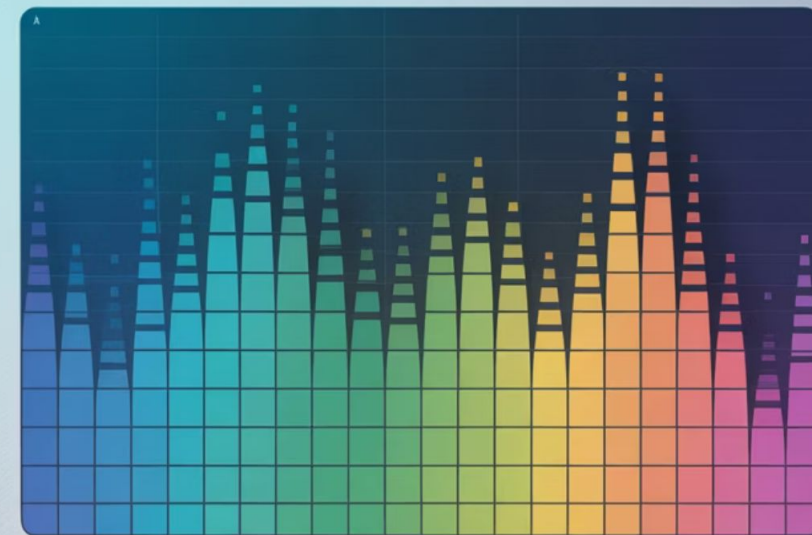


# Aplicar color a líneas y series

El argumento color permite especificar el color de una línea o marcador. Se pueden usar nombres de colores ('red', 'navy'), códigos hexadecimales ('#1f77b4') o valores RGB.

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [10, 20, 15],
color='darkorange')
ax.set_title('Color personalizado en línea')
plt.show()
```

También es posible usar paletas predefinidas como tab10, Set2 o incluso integrarse con librerías como Seaborn o estilos CSS para definir esquemas cromáticos consistentes.



**Matplotlib Color Customization  
Example**

# Estilos de línea y marcador

Matplotlib permite personalizar no solo el color de una línea, sino también su tipo de trazo y la forma de los puntos de datos. Estas opciones son fundamentales para diferenciar múltiples series en un mismo gráfico o para comunicar distintos estados o categorías. A continuación, se presentan las opciones más comunes:

Las líneas y los puntos pueden personalizarse con los argumentos `linestyle` y `marker`, respectivamente:

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [10, 20, 15],
linestyle='--', marker='o', color='purple')
ax.set_title('Línea discontinua con
marcadores')
plt.show()
```

| Estilo de línea (linestyle) | Descripción       |
|-----------------------------|-------------------|
| '-'                         | Línea continua    |
| '--'                        | Línea discontinua |
| '-.'                        | Línea punto-guión |
| '...'                       | Línea punteada    |

# Tipos de marcadores

| Marcador (marker) | Representación |
|-------------------|----------------|
| 'o'               | Círculo        |
| 's'               | Cuadro         |
| '^'               | Triángulo sup. |
| 'D'               | Diamante       |



# Combinación de estilos en una sola cadena

También es posible definir color, marcador y línea en un solo string:

```
ax.plot([1, 2, 3], [10, 20, 15], 'ro--') # rojo,  
círculos, línea discontinua
```

Esto es útil para gráficos rápidos, pero en proyectos estructurados se recomienda explicitar los parámetros para mayor legibilidad.

## Buenas prácticas

El uso de estilos debe estar al servicio de la comprensión, no de la decoración.

Algunas buenas prácticas incluyen:

- Usar paletas consistentes con el diseño corporativo o del informe: fortalece la identidad visual.
- Evitar colores muy similares para series distintas: puede generar confusión, especialmente para personas con dificultades visuales.
- Reservar marcadores para distinguir series que comparten colores: útil cuando se imprime en blanco y negro.
- No saturar visualmente el gráfico con demasiados estilos combinados: menos es más.





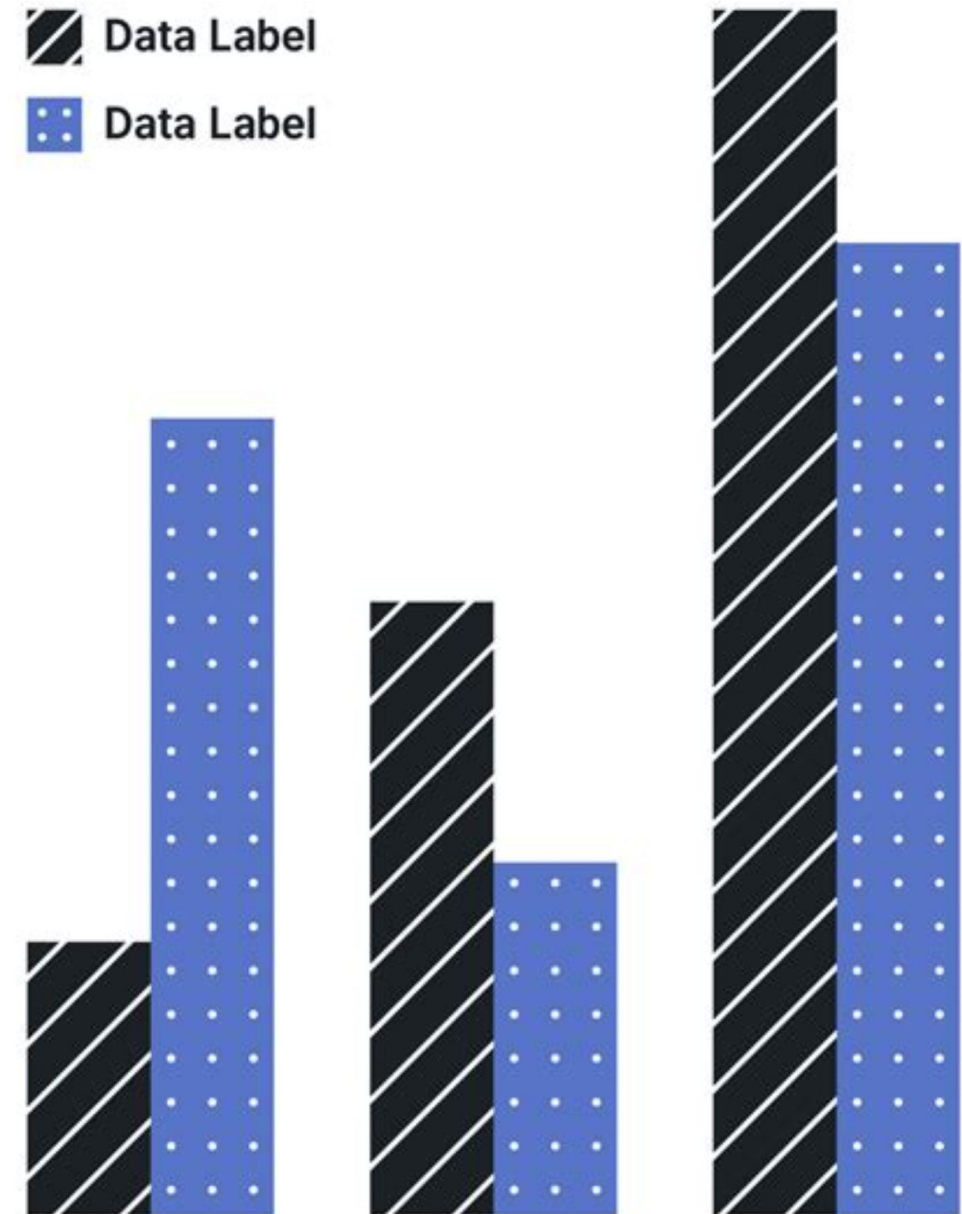
# Errores comunes y cómo evitarlos

| Error                                  | Causa probable                      | Solución recomendada                               |
|----------------------------------------|-------------------------------------|----------------------------------------------------|
| Colores demasiado similares            | Paleta automática poco contrastante | Definir colores explícitamente                     |
| Gráfico difícil de leer en escala gris | Abuso del color sin marcador        | Agregar marcadores distintivos por serie           |
| Estilos no documentados                | Uso de combinaciones sin leyenda    | Incluir siempre leyenda y comentarios en el código |

# Importancia del estilo gráfico

El estilo gráfico no es una cuestión estética superficial, sino una herramienta para transmitir significado. La correcta elección de colores, formas y estilos permite crear visualizaciones inclusivas, accesibles y eficaces.

En el siguiente apartado aprenderemos a rotular con precisión nuestros gráficos usando ticks, etiquetas y leyendas, elementos clave para la interpretación del lector.



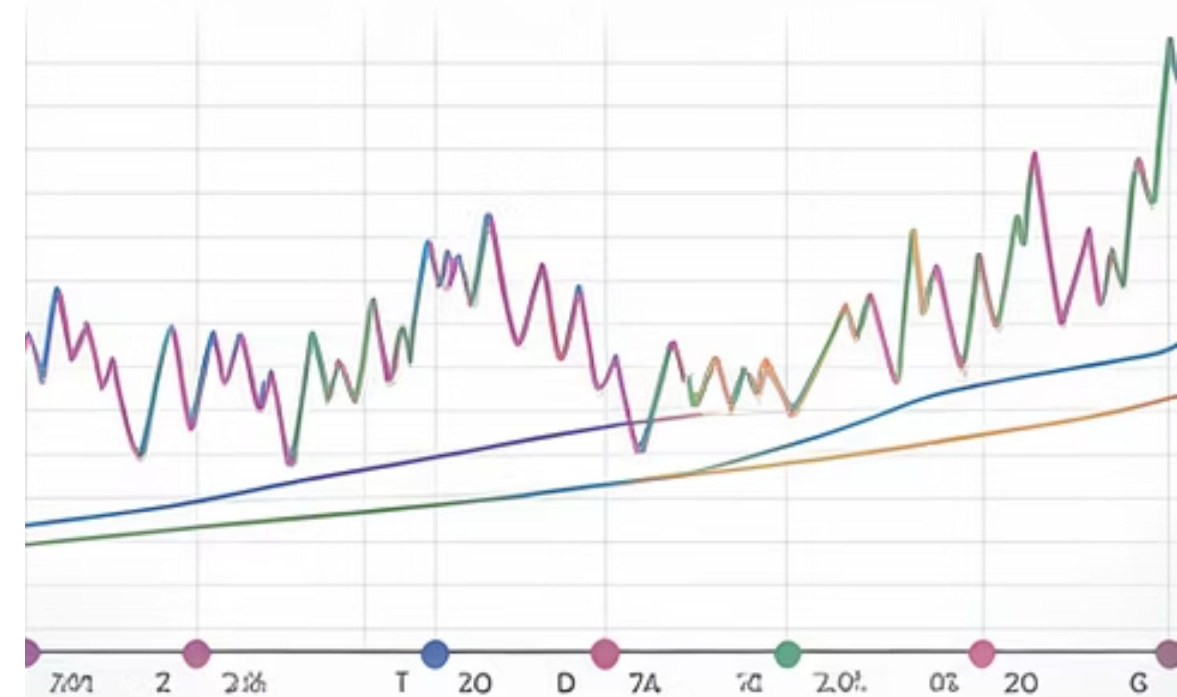
# Ticks, etiquetas y leyendas

**¿Alguna vez te ha costado entender un gráfico por falta de etiquetas o leyendas claras?**

Un gráfico bien diseñado no solo muestra datos: los explica. Para que la visualización sea comprensible, especialmente por audiencias no técnicas, es fundamental incorporar elementos descriptivos como etiquetas en los ejes, títulos, ticks ajustados y leyendas informativas. Estos componentes permiten interpretar correctamente qué se está mostrando, identificar tendencias y tomar decisiones basadas en datos.

En Matplotlib, estos elementos pueden personalizarse completamente para adaptarse a distintos contextos de uso (informes, presentaciones, dashboards, etc.).

## Matplotlib Labels & Legends Example

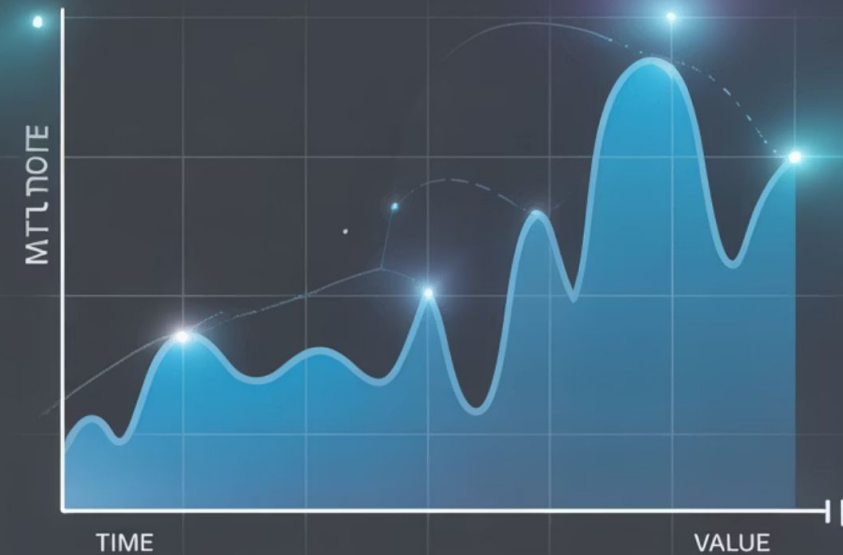


# Etiquetas y títulos

```
ax.set_title('Ventas mensuales por región')  
ax.set_xlabel('Mes')  
ax.set_ylabel('Ventas ($)')
```

Estas funciones permiten asignar textos explicativos a cada elemento del gráfico. Se pueden complementar con argumentos como fontsize, color, pad (espaciado) o loc (ubicación).

## Data Visualization



# Ticks

Los ticks son las marcas en los ejes que indican las divisiones del dato. Puedes personalizar sus valores, rotación o formato:

```
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(['Enero', 'Febrero',
'Marzo'], rotation=45)
```

También puedes usar `ax.tick_params()` para modificar tamaño, dirección y visibilidad de los ticks.

| Función                     | Uso                                       |
|-----------------------------|-------------------------------------------|
| set_xticks() / set_yticks() | Define la posición de los ticks           |
| set_xticklabels()           | Asigna etiquetas personalizadas           |
| tick_params()               | Controla apariencia (grosor, color, etc.) |

# Leyenda (legend)

Una leyenda ayuda a identificar qué representa cada serie de datos:

```
ax.plot(x, y1, label='Región Norte')  
ax.plot(x, y2, label='Región Sur')  
ax.legend(loc='upper left')
```

Puedes controlar su ubicación, borde, tamaño y si se superpone a otras partes del gráfico. Es importante no saturar de etiquetas y mantener la leyenda clara y ordenada.



# Buenas prácticas

Configurar correctamente etiquetas, ticks y leyendas asegura que la visualización sea comprensible y profesional. Algunas recomendaciones clave son:

-  **Evitar valores de ticks demasiado densos**  
Dificultan la lectura.
-  **Rotar etiquetas cuando ocupan mucho espacio horizontal**  
Mejora la legibilidad y evita solapamientos.
-  **Siempre incluir título y etiquetas de ejes**  
Sin contexto no hay comprensión.
-  **Ubicar la leyenda en zonas no obstructivas**  
Preferiblemente al margen o dentro de espacios libres.



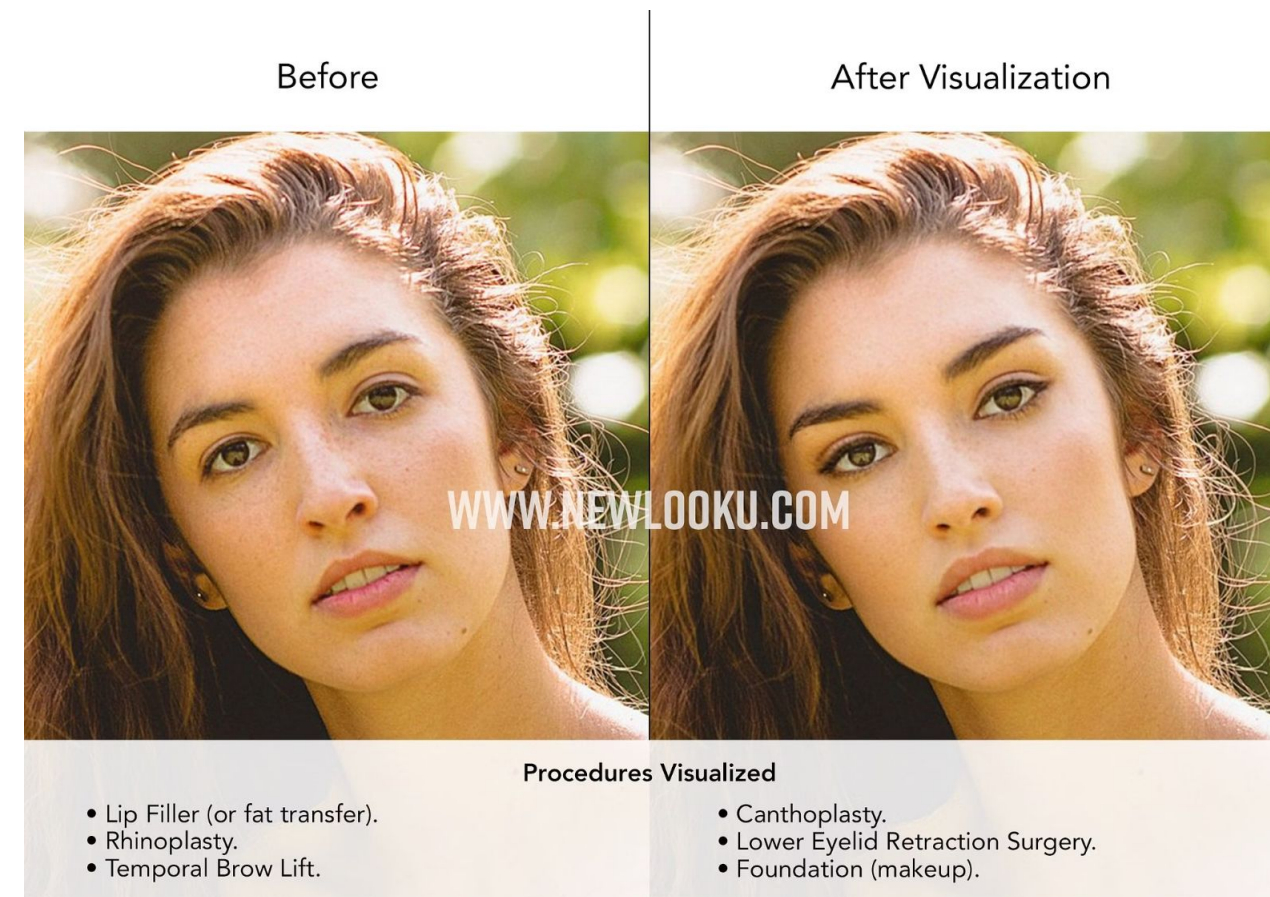
# Errores comunes y cómo evitarlos

| Error                       | Causa probable                                              | Solución recomendada                                        |
|-----------------------------|-------------------------------------------------------------|-------------------------------------------------------------|
| Leyenda sobre el gráfico    | loc mal definido o sin ajuste manual                        | Probar ubicaciones (loc) y usar <code>bbox_to_anchor</code> |
| Ticks solapados o ilegibles | Falta de rotación                                           | Agregar rotation en <code>set_xticklabels()</code>          |
| Ejes sin etiquetas          | Olvido de <code>set_xlabel</code> / <code>set_ylabel</code> | Incluir etiquetas descriptivas obligatoriamente             |

# Impacto de la personalización

Personalizar etiquetas, ticks y leyendas convierte un gráfico genérico en una visualización clara, precisa y útil. Estos pequeños ajustes tienen un gran impacto en la calidad de comunicación de los datos.

Con estas herramientas, completamos la base para generar visualizaciones profesionales en Matplotlib.



# Actividad guiada: Informe gráfico profesional para presentación ejecutiva



# Ejercicio

## *Informe gráfico profesional para presentación ejecutiva*

### Objetivo de la actividad

Aplicar los conocimientos adquiridos sobre personalización de gráficos en Matplotlib para crear una visualización clara, escalable y presentable para contextos profesionales, incorporando tamaño adecuado, estilos personalizados y elementos descriptivos.

### Situación problema

Formas parte del equipo de inteligencia comercial de una empresa de tecnología. Debes presentar los resultados trimestrales de ventas en cuatro productos diferentes, cada uno representado por una serie de datos. La presentación se realizará ante la gerencia general, por lo que necesitas entregar una visualización profesional, clara, bien rotulada, con estilos coherentes y lista para exportar a un informe ejecutivo.



# Ejercicio

## Instrucciones

### Crea un script o celda de notebook en Python que:

- Cree una figura de tamaño adecuado (figsize) y subgráficos si es necesario.
- Ingrese al menos cuatro series de datos (pueden ser simuladas).
- Aplique estilos personalizados por serie: color, tipo de línea y marcador.
- Incluya un título general, etiquetas en los ejes, ticks legibles y una leyenda clara.
- Ajuste los márgenes con `tight_layout()` y `subplots_adjust()` si es necesario.
- Exporte el gráfico en formato .png con márgenes adecuados.



# Ejercicio

## Instrucciones

## Código base sugerido

```
import matplotlib.pyplot as plt
meses = ['Ene', 'Feb', 'Mar']
producto_a = [200, 220, 250]
producto_b = [180, 210, 190]
producto_c = [150, 180, 160]
producto_d = [100, 140, 130]
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(meses, producto_a, label='Producto A', linestyle='-', marker='o', color='blue')
ax.plot(meses, producto_b, label='Producto B', linestyle='--', marker='s', color='green')
ax.plot(meses, producto_c, label='Producto C', linestyle='-.', marker='^', color='orange')
ax.plot(meses, producto_d, label='Producto D', linestyle=':', marker='D', color='red')
ax.set_title('Ventas trimestrales por producto')
ax.set_xlabel('Mes')
ax.set_ylabel('Ventas ($)')
ax.set_xticks(range(len(meses)))
ax.set_xticklabels(meses)
ax.legend(loc='upper left')
plt.tight_layout()
plt.savefig('informe_ventas.png', bbox_inches='tight')
plt.show()
```

Asegúrate de que el gráfico sea claro, ordenado y comprensible sin explicación verbal.

# Actividad práctica autónoma: Visualización personalizada para informe ejecutivo





# Ejercicio

## *Visualización personalizada para informe ejecutivo*

### Objetivo de la actividad

Aplicar los principios de personalización avanzada de Matplotlib (tamaño, estilo, etiquetas, leyendas) para diseñar un gráfico profesional que sea exportable, visualmente claro y comunicativamente eficaz en un entorno ejecutivo.

### Situación problema

Has sido contratado como analista freelance para preparar una visualización ejecutiva que será incluida en un informe de resultados trimestrales de una empresa. El equipo te entregó una tabla con los datos de ventas mensuales de tres regiones (Norte, Centro y Sur), y tu trabajo consiste en construir un gráfico de líneas claro y profesional, con estilo y elementos descriptivos personalizados.

La visualización debe poder insertarse directamente en el informe sin requerir ajustes, por lo tanto, se espera que domines aspectos como el tamaño, los márgenes, la rotulación, los colores y la ubicación de la leyenda.



# Ejercicio

## Instrucciones

1. Descarga el archivo: ventas\_regionales\_autonomo.csv
2. Abre tu entorno de trabajo en Python y carga los datos:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("ventas_regionales_autonomo.csv")
print(df)
```

**Crea un gráfico de líneas personalizado con los siguientes requerimientos:**

- Tamaño total de la figura: (10, 5)
- Estilo diferente para cada serie (línea, color y marcador)
- Título: "Ventas mensuales por región"
- Etiquetas: eje x "Meses", eje y "Ventas (\$)"
- Leyenda ubicada en la esquina inferior derecha
- Exporta el gráfico en formato .png usando bbox\_inches='tight'

**{desafío}**  
**latam\_**





# Revisión del gráfico



**¿Las etiquetas y títulos están legibles?**

Verifica que el tamaño de fuente sea adecuado y que no haya solapamientos.



**¿Los estilos permiten diferenciar claramente cada región?**

Comprueba que los colores, líneas y marcadores sean suficientemente distintos.



**¿La leyenda se ubica correctamente y no interfiere con el gráfico?**

Asegúrate de que la leyenda sea visible pero no obstruya datos importantes.

## Ejemplo de código base

```
meses = df["Mes"]
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(meses, df["Norte"], label="Región Norte",
linestyle='-', marker='o', color='navy')
ax.plot(meses, df["Centro"], label="Región Centro",
linestyle='--', marker='s', color='green')
ax.plot(meses, df["Sur"], label="Región Sur", linestyle='-.',
marker='^', color='orange')
ax.set_title("Ventas mensuales por región", fontsize=14)
ax.set_xlabel("Meses", fontsize=12)
ax.set_ylabel("Ventas ($) ", fontsize=12)
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig("grafico_ventas_personalizado.png",
bbox_inches="tight")
plt.show()
```

# Resumen



## Tamaño y posición

Aprendimos a definir y ajustar el tamaño y la posición de los gráficos para distintos entornos de presentación.



## Estilos visuales

Aplicamos estilos visuales personalizados usando colores, tipos de línea y marcadores para mejorar la claridad de las visualizaciones.



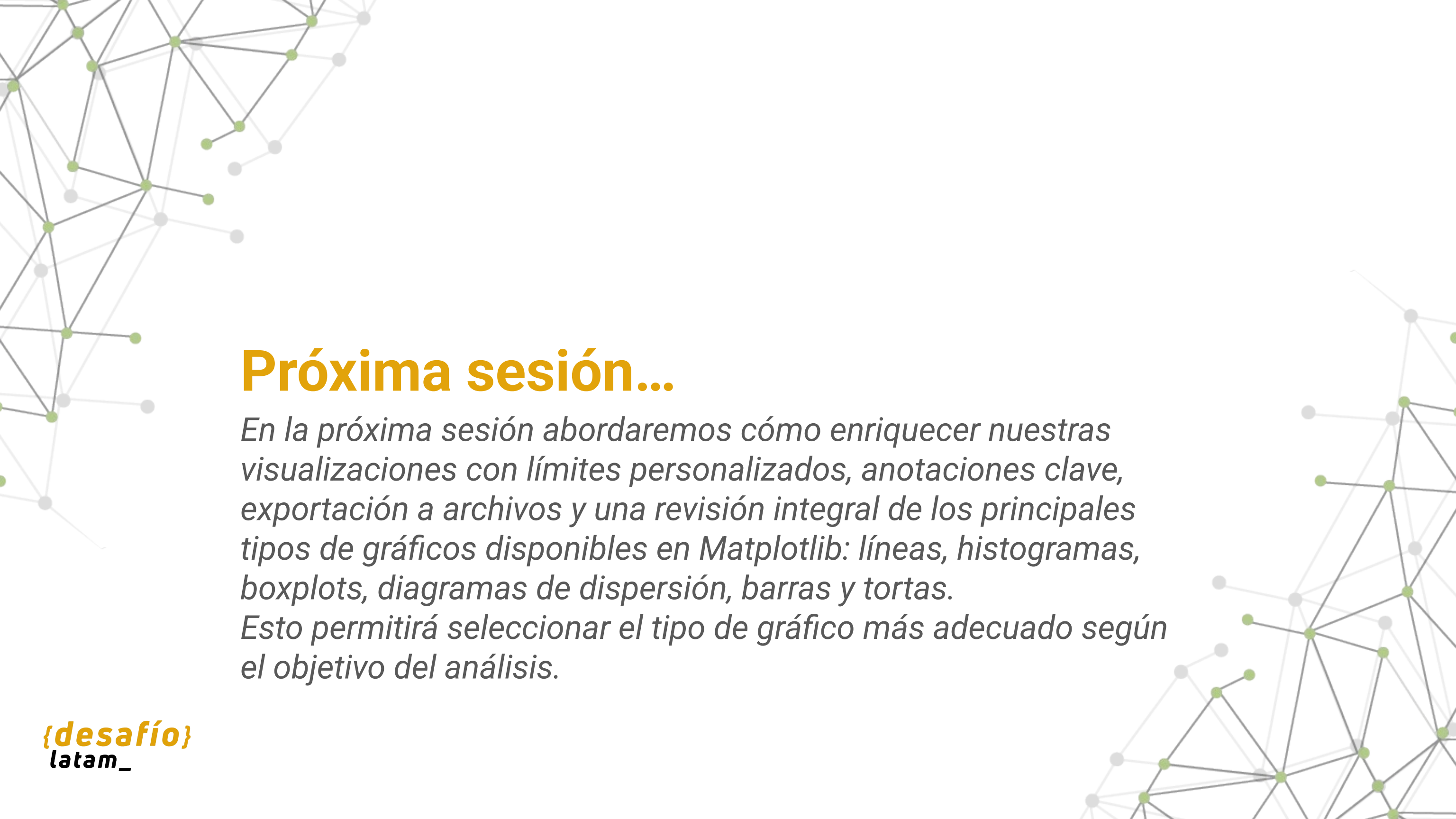
## Etiquetas y leyendas

Configuramos etiquetas, ticks y leyendas para comunicar de forma precisa y profesional el contenido del gráfico.



## Buenas prácticas

Identificamos errores comunes y aplicamos buenas prácticas en la construcción de gráficos con Matplotlib.



## Próxima sesión...

*En la próxima sesión abordaremos cómo enriquecer nuestras visualizaciones con límites personalizados, anotaciones clave, exportación a archivos y una revisión integral de los principales tipos de gráficos disponibles en Matplotlib: líneas, histogramas, boxplots, diagramas de dispersión, barras y tortas. Esto permitirá seleccionar el tipo de gráfico más adecuado según el objetivo del análisis.*

# Preguntas de cierre

**¿Cómo puedes controlar la proporción de un gráfico para que se adapte a un informe impreso?**

Utiliza el parámetro `figsize` al crear la figura, definiendo dimensiones en pulgadas que sean proporcionales al espacio disponible en el informe.

**¿Qué diferencias clave existen entre personalizar una línea con parámetros separados vs. en una cadena combinada?**

Los parámetros separados (`color='red', linestyle='--', marker='o'`) ofrecen mayor legibilidad y control, mientras que la cadena combinada (`'ro--'`) es más compacta pero menos explícita y puede dificultar el mantenimiento del código.

**¿Cuándo es recomendable usar `tight_layout()` y qué problema ayuda a resolver?**

Es recomendable usar `tight_layout()` cuando se agregan muchos elementos como títulos, etiquetas o leyendas, ya que ayuda a evitar solapamientos y cortes de texto ajustando automáticamente los márgenes.

**¿Qué estrategia aplicarías si los ticks de un eje se superponen con las etiquetas?**

Rotaría las etiquetas con `set_xticklabels(['Etiqueta1', 'Etiqueta2'], rotation=45)`, reduciría su número seleccionando solo los más importantes, o aumentaría el tamaño de la figura para dar más espacio.

**¿Cómo afecta una leyenda mal posicionada en la interpretación del gráfico?**

Una leyenda mal posicionada puede ocultar datos importantes, confundir al lector sobre la correspondencia entre series y etiquetas, o dificultar la lectura general del gráfico, comprometiendo su efectividad comunicativa.